Bone X-Ray Fracture Detection Project
Complete End-to-End Technical Guide

This document describes the fracture detection project in full detail, including the complete processing pipeline, exact responsibilities of each role, code structure, data flow, and implementation expectations. The intention is that contributors can work independently using this document as the primary reference.

Project Resources

Dataset:
https://www.kaggle.com/datasets/vuppalaadithyasairam/bone-fracture-detection-using-xrays?resource=download

Repository:
https://github.com/vishwaraj25/xray-fracture-detection/tree/main

Overall Project Workflow

The system follows a strict linear pipeline. Each stage produces output that becomes the input for the next stage. No stage skips or bypasses another.

X-ray Image → Preprocessing → Feature Extraction → Dataset Construction → SVM Training → Prediction

Project Folder Structure

```
xray-fracture-detection/ data/ fracture_dataset/ fractured/ not_fractured/ src/ preprocessing.py feature_extraction.py build_dataset.py train_svm.py run_pipeline.py venv/ requirements.txt README.md
```

Role 1: Image Preprocessing and Feature Extraction

This role converts raw X-ray images into structured numerical representations. It is responsible for both image cleanup and feature computation.

Implementation Details

1. Read image from disk using cv2.imread(path)
2. Convert image to grayscale using cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
3. Resize image to a fixed resolution (for example 224×224) using cv2.resize
4. Apply Gaussian blur using cv2.GaussianBlur to reduce noise
5. Apply edge detection using cv2.Canny to emphasize bone boundaries
6. Compute HOG features using skimage.feature.hog
7. Compute LBP features using skimage.feature.local_binary_pattern
8. Concatenate HOG and LBP into a single 1D NumPy array

Expected Output: A NumPy array of fixed length representing one image.

While implementing this, it may be useful to think in terms of: a function that accepts an image path and returns a feature vector, with no side effects and no model logic.

Role 2: Dataset Construction

This role is responsible for transforming the entire image dataset into a machine-learning-ready format. It acts as the bridge between image processing and model training.

Implementation Details

1. Traverse the dataset directories using os.listdir or os.walk
2. For each image file, determine its label based on folder name
3. Call the feature extraction function for every image
4. Append each feature vector to a Python list
5. Append corresponding labels to a separate list
6. Convert feature and label lists into NumPy arrays
7. Verify array shapes: X → (num_samples, num_features), y → (num_samples,)
8. Persist X and y to disk using numpy.save

Expected Output: Two files saved to disk, typically features.npy and labels.npy.

At this stage, correctness and consistency are more important than speed. It can help to print shapes, sample values, and confirm that labels align with their corresponding feature vectors.

When writing this code, it is often useful to think in terms of: a script that takes a folder path and produces two NumPy files that can be loaded later without modification.

Role 3: Model Training and Prediction

This role focuses on learning a decision boundary between fractured and non-fractured samples using a classical machine learning classifier.

Implementation Details

1. Load features.npy and labels.npy using numpy.load
2. Split data into training and testing sets using train_test_split
3. Initialize an SVM classifier using sklearn.svm.SVC with default parameters
4. Fit the model on the training data
5. Generate predictions on the test data
6. Print or log prediction results for verification
7. Save the trained model to disk using joblib.dump
8. Reload the saved model and run prediction on a new sample

Expected Output: A trained SVM model file and the ability to output a fracture or non-fracture prediction.

This stage is intended to follow a standard classification workflow without hyperparameter tuning. Following a reference implementation of scikit-learn SVM classification is sufficient.

It can be useful to approach this stage by considering: a script that loads prepared data, trains once, saves the model, and reuses it for inference.

Version Control and Collaboration Guidelines

1. Each contributor works only within their assigned source files
2. Dataset images are kept local and are never committed to GitHub

3. All changes are committed incrementally with descriptive messages
4. Branches are used for individual development and merged when stable

The project structure prioritizes clarity, modularity, and predictable integration.